



Graph Theory

Group Homework - Minimum Spanning Tree Algorithms

Work realised by Melvyn, Mikal, Denis and Hugo

Teacher: Ilham Gurat Adillion

Date: 09/2026

Level: 3rd Semester

Summary

Summary	2
1. Introduction	3
2. Prim's Algorithm Implementation	4
3. Kruskal's Algorithm Implementation	6
3.1. Sorted Edges List	6
3.2. Algorithm Execution Table	6
4. Alternative MST Algorithm (Borůvka's Algorithm)	7
Algorithm Principle	7
Execution Trace on the Graph	7
Key Advantage	7
5. Edge/Node Failure Simulation	7
Scenario	7
Adaptation Mechanism	7
Results and Comparison	8
Conclusion of Simulation	8
6. Conclusion & Repository Reference	8

1. Introduction

This report details the implementation and comparative analysis of Minimum Spanning Tree (MST) algorithms applied to a specific graph network, adapted from MIT Urban Operations Research. The primary objective of this assignment is to determine the most cost-effective routing that connects all nodes within the network without forming any cycles.

To achieve this, we analyze a weighted, undirected graph consisting of 7 nodes (A through G) and 12 distinct edges. We will apply and trace three different algorithms to solve the MST problem: Prim's Algorithm, Kruskal's Algorithm, and Borůvka's Algorithm (chosen as our independent third algorithm). Furthermore, this report includes a simulation of a network disruption—specifically an edge failure—to demonstrate the resilience and adaptability of our implemented solution in recalculating the optimal spanning tree.

The initial state of the graph network, including all nodes and corresponding edge weights, is illustrated below.

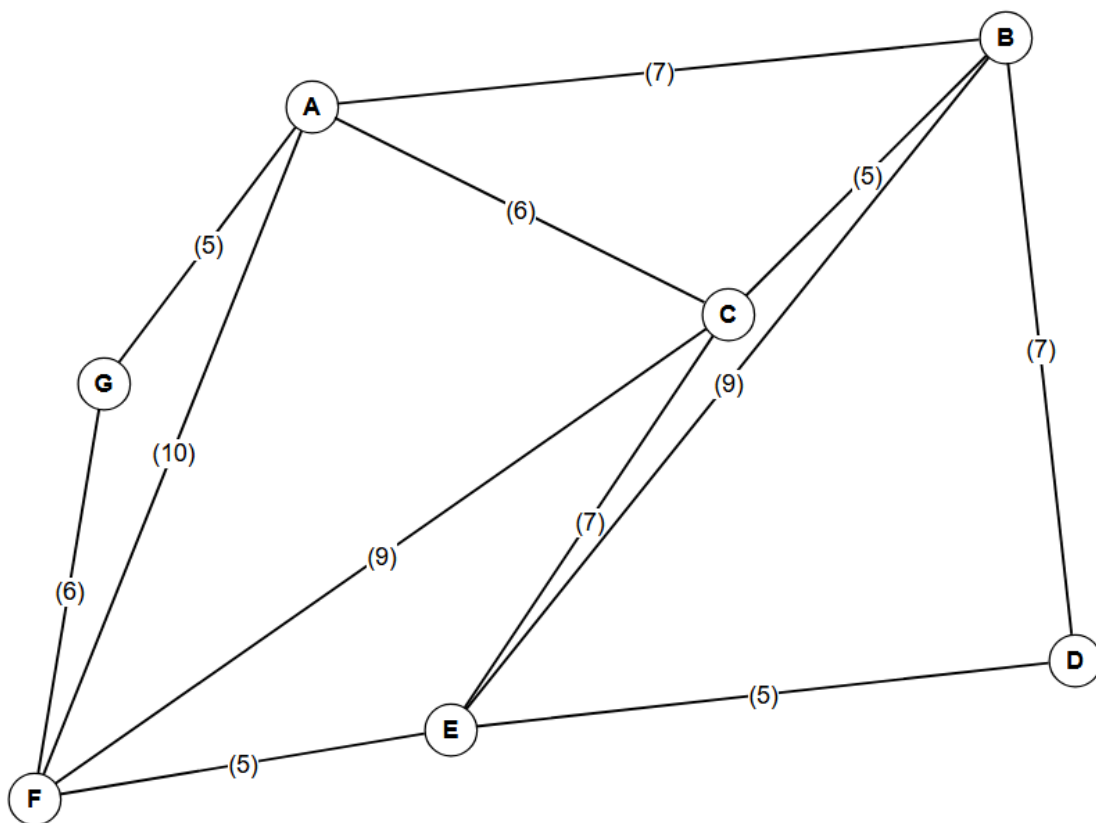
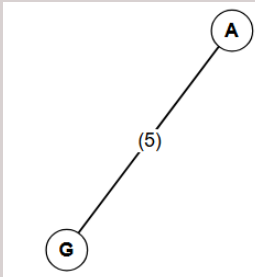
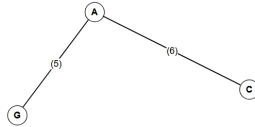
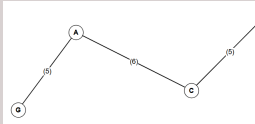


Figure 1: Initial Graph Network with Edge Weights

2. Prim's Algorithm Implementation

Step	U	Edge(u, v)	V - U	Graph
0	{}	-	{A, B, C, D, E, F, G}	-
1	{A}	(A-G) = 5 V (A-C) = 6 (A-B) = 7 (A-F) = 10	{B, C, D, E, F, G}	
2	{A, G}	(A-C) = 6 V (G-F) = 6 (A-B) = 7 (A-F) = 10	{B, C, D, E, F}	
3	{A, G, C}	(C-B) = 5 V (G-F) = 6 (A-B) = 7 (C-E) = 7 (C-F) = 9 (A-F) = 10	{B, D, E, F}	
4	{A, G, C, B}	(A-B) = 7 cycle (G-F) = 6 V (C-E) = 7 (B-D) = 7 (C-F) = 9 (B-E) = 9	{D, E, F}	

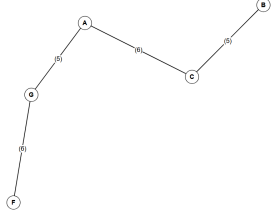
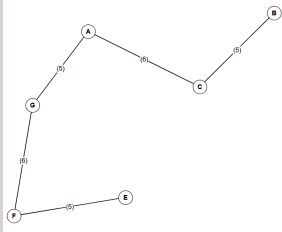
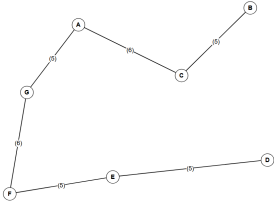
(A-F) = 10		
5	{A, G, C, B, F}	{D, E} (A-B) = 7 cycle (C-F) = 9 cycle (A-F) = 10 cycle (F-E) = 5 V (C-E) = 7 (B-D) = 7 (B-E) = 9
		
6	{A, G, C, B, F, E}	{D} (A-B) = 7 cycle (C-F) = 9 cycle (A-F) = 10 cycle (C-E) = 7 cycle (B-E) = 9 cycle (E-D) = 5 V (B-D) = 7
		
7	{A, G, C, B, F, E, D}	{} (A-B) = 7 cycle (C-F) = 9 cycle (A-F) = 10 cycle (C-E) = 7 cycle (B-E) = 9 cycle (B-D) = 7 cycle
		

Table 2 : Prim's Algorithm Implementation

3. Kruskal's Algorithm Implementation

3.1. Sorted Edges List

Before starting, all edges are sorted in non-decreasing order of their weight: (A-G): 5, (C-B): 5, (F-E): 5, (E-D): 5, (A-C): 6, (G-F): 6, (A-B): 7, (C-E): 7, (B-D): 7, (F-C): 9, (E-B): 9, (A-F): 10.

(Note: Edges with the same weight can be processed in any order. The result will still be a valid Minimum Spanning Tree).

3.2. Algorithm Execution Table

Step	Edge Considered	Weight	Action / Status	Current Forest (Disjoint Sets)
0	-	-	Initialization	{A}, {B}, {C}, {D}, {E}, {F}, {G}
1	(A-G)	5	Added (V)	{A, G}, {B}, {C}, {D}, {E}, {F}
2	(C-B)	5	Added (V)	{A, G}, {B, C}, {D}, {E}, {F}
3	(F-E)	5	Added (V)	{A, G}, {B, C}, {D}, {E, F}
4	(E-D)	5	Added (V)	{A, G}, {B, C}, {D, E, F}
5	(A-C)	6	Added (V)	{A, G, B, C}, {D, E, F}
6	(G-F)	6	Added (V)	{A, G, B, C, D, E, F}
7	(A-B)	7	Rejected (Cycle)	{A, G, B, C, D, E, F}
8	(C-E)	7	Rejected (Cycle)	{A, G, B, C, D, E, F}
9	(B-D)	7	Rejected (Cycle)	{A, G, B, C, D, E, F}
10	(F-C)	9	Rejected (Cycle)	{A, G, B, C, D, E, F}
11	(E-B)	9	Rejected (Cycle)	{A, G, B, C, D, E, F}
12	(A-F)	10	Rejected (Cycle)	{A, G, B, C, D, E, F}

Table 3 : Kruskal's Algorithm Implementation

4. Alternative MST Algorithm (Borůvka's Algorithm)

Algorithm Principle

Unlike Prim and Kruskal, Borůvka's algorithm builds the Minimum Spanning Tree (MST) by initializing every vertex as a standalone component. In each iteration, it simultaneously finds and adds the minimum-weight outgoing edge for every component until a single, unified tree remains.

Execution Trace on the Graph

- **Iteration 1 (Initial Grouping):** The algorithm simultaneously selects the lowest-cost edge for each individual node. It adds edges (A-G), (B-C), (F-E), and (E-D), all sharing the minimum weight of 5. This forms three disjoint components: {A, G}, {B, C}, and {D, E, F}.
- **Iteration 2 (Final Merger):** It then finds the minimum outgoing edges to connect these three newly formed components. It selects (A-C) and (G-F), both with a weight of 6. This successfully merges all components into the final MST.

Key Advantage

Because edge selections happen simultaneously across components, this algorithm is inherently parallelizable. This makes it particularly effective for distributed computing and extremely large networks where sequential processing would be inefficient.

5. Edge/Node Failure Simulation

Scenario

To test the robustness of the network and the adaptability of our algorithms, we simulate the sudden failure of a critical link. In this scenario, we assume that the edge connecting node **B** and node **C** (which has a weight of 5 and is part of the original optimal path) fails and can no longer be traversed.

Adaptation Mechanism

When a failure occurs, the algorithm must dynamically recalculate the Minimum Spanning Tree using the updated graph constraints.

1. The failed edge (B-C) is permanently removed from the graph's list of available edges.
2. The algorithm (e.g., Kruskal's algorithm) is re-executed on the modified network. It evaluates the remaining edges to find the most cost-effective way to reconnect the resulting disjointed components without forming cycles.

Results and Comparison

- **Original State:** Before the failure, the MST utilized the B-C edge to connect node B to the rest of the network through node C. The total cost of the original MST was **32**.
- **Adapted State:** With B-C unavailable, node B becomes temporarily isolated from the spanning tree. The algorithm evaluates alternative routes and identifies the A-B edge (weight 7) as the next cheapest valid connection (it could also pick B-D, which also has a weight of 7, depending on sorting logic).
- **New MST Configuration:**
 - (A-G) = 5
 - (D-E) = 5
 - (E-F) = 5
 - (A-C) = 6
 - (F-G) = 6
 - **(A-B) = 7** (*New edge selected to reconnect node B*)
- **New Total Cost:** The total weight of the newly adapted MST increases to **34**.

Conclusion of Simulation

The algorithm successfully adapts to the simulated edge failure by identifying and utilizing the next optimal path. Because the cheapest link to node B (weight 5) was destroyed, the network incurs a routing penalty of +2 to its total spanning cost. This demonstrates the algorithm's capability to guarantee network connectivity, automatically self-healing as long as alternative physical paths exist within the graph.

6. Conclusion & Repository Reference

Through this assignment, we successfully applied and compared Prim's, Kruskal's, and Borůvka's algorithms on a weighted, undirected graph network. While their execution methodologies differ significantly—ranging from node-centric growth (Prim) to edge-centric sorting (Kruskal) and parallel component merging (Borůvka)—all three approaches reliably computed the identical Minimum Spanning Tree with an optimal total routing cost of 32.

Furthermore, the edge failure simulation highlighted the practical resilience of these algorithmic methods. By artificially severing a critical link, we demonstrated the algorithms' ability to dynamically recalculate the topology, ensuring the network self-heals and maintains full connectivity with the next most cost-effective routing configuration.

Repository Reference The complete Python source code containing the implementation of all three MST algorithms, as well as the edge failure simulation, is hosted on GitHub. You can review the code via the following link:

[GitHub Repository - MST Algorithms Implementation](#)